

Domain 4: Prompt Engineering & Structured Output

Domain 4 makes up 20% of the scored content, and it's where you learn to get Claude to produce exactly the output you need, in exactly the shape your system can use. You'll cover how to engineer context and design prompts that steer Claude toward the response you're after, how to define JSON schemas that lock output into a predictable structure, how to pull clean structured data out of messy text with extraction patterns, how to use few-shot examples to show Claude the transformation you want, and how to build validation loops that catch a bad output and recover before it reaches the next step.

Think of prompting as the steering wheel between what you want and what Claude actually returns. When prompts are vague and output runs loose, you get inconsistent answers, formats that break your parser, and pipelines that fail in ways you can't predict. Get the prompt and the schema right, and everything downstream can trust what it receives.

A. Foundations of Prompt Engineering for Production

Prompt engineering is the work of writing the input to Claude so the output reliably does what you need. A casual user is satisfied with one good answer. A production system has a higher bar: the same prompt may run thousands of times against very different inputs, and it has to behave the same way each time.

Claude does excellent work when the instructions are clear, and it fills in gaps with its own assumptions when the instructions are vague. So the more precisely you say what you want, the less the model has to guess, and the more consistent the output becomes. Prompt engineering is also the fastest and cheapest lever available. It changes only the input you send, not the model itself. The instructions stay readable, and they keep working as the model improves. That is why it is usually the first tool to reach for, and why knowing its limits matters as much as knowing its techniques.

Key Terms

- **Prompt engineering** is designing the input so the model reliably produces what you want.
- **System prompt** is the top-level instruction block that sets the model's role, rules, and output expectations.
- **Probabilistic** means the behavior is about likelihood. A good prompt makes the right answer far more likely, but never certain.
- **Deterministic mechanism** is a control that always holds, such as a code gate, a hook, or strict schema enforcement.

What is Prompt Engineering?

It is writing instructions, context, and examples that point the model at the result you want. Unlike traditional software where you write code that executes exactly as written, prompt engineering works by shaping the model's understanding of what a good response looks like. The model brings its own reasoning to every request, so your job is to direct that reasoning, not replace it.

How It Works

Prompt engineering works on the input you send, not on the model's internal weights, so changes take effect immediately. There is no retraining, no deployment pipeline, and no waiting. You write a new prompt, test it, and adjust. This makes it the fastest and cheapest intervention available. Before reaching for fine-tuning, retrieval, or additional tooling, a better prompt often solves the problem at a fraction of the cost and effort.

It is also iterative by nature. A prompt that works on one input may fail on another, so testing across varied examples is part of the process, not an afterthought. The goal is not a prompt that works once but one that holds across the range of inputs the system will actually see.

- Write the prompt with a clear goal in mind.
- Test it against real or representative inputs, not just the easy cases.
- Identify where it fails and why.
- Refine the instruction, example, or constraint that caused the failure.
- Repeat until the output is consistent across the full range of expected inputs.

Zero-Shot and Few-Shot Prompting

Type	Instruction	Example Provided
Zero-shot	"Flag the important issues"	The model decides what important means. That decision changes from run to run.
Few-shot	"Review this code"	The model picks a scope. A re-run picks a different one.

- **Zero-shot** works well for simple, well-defined tasks where the expected output is obvious.
- **Few-shot** works better for complex formats, judgment calls, or cases where the correct output is hard to describe in words.

The key difference is not the answer but the reliability. Both return the correct label in this case. The advantage of few-shot learning shows when the task is more ambiguous or the output format is more specific. Without examples, the model decides what "positive" and "negative" look like on its own. With examples, you have already shown it.

How Claude Interprets Instructions

Claude does not read only the sentence you care about. It reads the whole prompt at once, including the system prompt, the descriptions of any tools, and any examples, and it forms a single overall sense of what you want from all of it together.

- It responds to what the prompt actually says, not to the intention in your head.
- Because every part counts, a stray phrase can push behavior in a direction you did not plan.
- Contradictory instructions pull the model two ways and lower reliability.
- A single keyword in a system prompt can even bias which tool the model chooses, so word system prompts carefully.

The Context Window

The context window is the boundary of what Claude can see at one time. Everything outside it does not exist for that request. For long documents, long conversation histories, or large tool definitions, relevant instructions can get crowded out by other content.

- In very long prompts, attention can spread thin. Key instructions that would be obvious in a short prompt can lose influence when surrounded by many other tokens.
- What fits in the window and where it sits both affect how reliably instructions are followed.
- For critical instructions in long prompts, repetition is a valid strategy. Stating the same rule at the start and again near the end costs tokens but improves reliability.

References

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/be-clear-and-direct>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/use-xml-tags>

Intent versus Interpretation

Whenever your prompt leaves something unsaid, the model makes a reasonable guess to fill the gap. Its guess may not match yours. That space between what you meant and how the model read it is where most inconsistency comes from.

Why the Gap Exists

You have full context for your request: the audience, the constraints, the definition of success. The model has none of that unless you provide it. When details are missing, the model constructs a plausible version of your situation and responds to that version. Nothing in the response signals that any assumption was made.

Common Forms of Mismatch

Type	Example	What Goes Wrong
Undefined criteria	"Flag the important issues"	The model decides what important means. That decision changes from run to run.
Ambiguous scope	"Review this code"	The model picks a scope. A re-run picks a different one.
Unstated audience	"Write an explanation"	Model defaults to a general audience that may not fit your actual reader.
Vague length	"Keep it concise"	The model chooses a length. Concise means different things to different people.

Each of these mismatches shares the same root cause: the prompt used a word that felt precise to the writer but left the model room to decide. "Important," "review," "explanation," and "concise" are all judgment calls, not instructions. The model fills that judgment with a reasonable default, and that default is not stable across runs.

The fix is the same in every case: replace the evaluative word with a concrete rule. Define what counts as important. Specify which kind of review you want. Name the audience. Give a word or sentence count.

Once the judgment call is removed from the model's side and placed in the prompt, the output stops varying.

Closing the Gap

Read your own prompt and ask: if someone read only these words with no access to my context, what assumptions would they have to make? Each assumption is a gap worth closing.

Vague	Precise
"Flag the important issues"	"Flag issues that would cause data loss or service unavailability"
"Review this code"	"Review for SQL injection and input validation only"
"Write an explanation"	"Write two paragraphs for a non-technical project manager"
"Keep it brief"	"Respond in no more than three sentences"

The Role of Specificity

- Specific: Flag a code comment only when the behavior it claims contradicts what the code actually does.
- Vague: Check that comments are accurate.

The specific version gives the model a clear test it can apply the same way every time. Words like "carefully" or "thoroughly" sound reassuring but change nothing, because they do not change the test the model is applying.

References

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/be-clear-and-direct>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/increase-consistency>

Prompt-Level Problems vs. Architectural Problems

Concept	Best Used For	Key Benefit	Common Exam Trap
Prompt-level fix	Clarity, consistency, formatting, ambiguous cases	Fast, cheap, no new infrastructure	Used where a hard guarantee is actually required
Architectural fix	Guaranteed ordering, required compliance, schema enforcement	Always holds, and you can verify it	Reached for when a clearer prompt would have done

A prompt-level fix works by reducing ambiguity. When output is inconsistent because the model is guessing at scope, format, or criteria, a clearer prompt removes that guesswork. This is the right tool for most problems and should always be the first thing you try.

An architectural fix works by removing the model's discretion entirely. A code gate, a schema validator, or a required pipeline step does not ask the model to comply. It enforces the rule regardless of what the model produces. This is the right tool when a rule must hold every single time without exception.

The exam trap in both cases is reaching for the wrong one. Using a prompt where a guarantee is required leaves a gap that will eventually fail. Adding infrastructure where a clearer instruction would have worked adds cost and complexity for no gain. The deciding question is simple: can this ever be allowed to fail? If yes, fix the prompt. If not, enforce it structurally.

The Limits of Prompting

A prompt instruction changes the odds, but it never reaches certainty. It can make a mistake rare, but not impossible.

- When a rule must always hold (for example: verify identity before issuing a refund), you need a deterministic mechanism, not a firmer instruction.
- Saying "always do this" more forcefully does not turn a probability into a guarantee.

WORKED EXAMPLE:

Vague: Review this code and flag anything important.

Explicit: Flag only (1) bugs that change behavior and (2) security issues. Do not flag style or formatting. For each finding, give: location, issue, severity (critical / major / minor), and a suggested fix.

What this shows: the explicit version names what to report, what to skip, and the exact shape of each finding. That removes the guesswork that makes the vague version inconsistent.

Common Mistakes

- Treating a prompt as a guarantee for a rule that must always hold.
- Adding infrastructure when the real problem was just an unclear instruction.
- Piling on vague qualifiers like "be careful" and expecting precision to improve.
- Writing system prompt instructions that quietly contradict the tool descriptions.

References

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/be-clear-and-direct>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

B. Designing Prompts with Explicit Criteria

Explicit criteria are concrete rules that state exactly what counts as a result worth reporting and what does not. Instead of leaving the model to decide what matters, you decide and write it down. This turns a fuzzy instruction into a clear, testable boundary.

Explicit Criteria vs. General Guidance

The core problem with vague instructions is not that the model ignores them. It is that the model follows them using its own judgment, and that judgment is not stable. Explicit criteria replace judgment with rules. General guidance leaves the judgment in place and just adds a modifier to it.

Concept	Best Used For	Key Benefit	Common Exam Trap
Explicit categorical criteria	Deciding what is and is not reportable	Consistent, predictable output	Mistaken for needing extra infrastructure
General guidance (be conservative)	A quick, low-effort instruction	Easy to write	Believed to improve precision when it does not

Why "Be Conservative" Does Not Work

Telling the model to be conservative or to only report high-confidence findings sounds like it should reduce noise, but it usually does not.

- The model is often already confident in the very cases it gets wrong, so asking for more confidence does not filter those out.
- The instruction never says where the line actually is, so the model still has to guess.
- Filtering by a confidence score is unreliable because the model's confidence is poorly calibrated on hard cases.
- Naming categories of what to report and what to ignore gives a concrete line the model can apply the same way every time.

Severity Classification

What is a severity level? It is a named tier, such as critical, major, or minor, with a stated condition for each. It replaces a vague label with a rule.

Anchoring severity with concrete examples: Attach a short example to each tier. Without examples, the same issue can land in major on one run and minor on the next.

False Positives and User Trust

- **The cost of false positives:** A category that often fires on non-issues poisons trust in the whole tool. Once people learn to ignore the noisy category, they start ignoring the accurate ones too.
- **Category suppression:** The practical recovery is to temporarily switch off the noisy category to restore trust, improve its criteria, and only then turn it back on.

WORKED EXAMPLE

Report:

- SQL injection or unsanitized input -> severity: critical
- A logic bug that changes the output -> severity: major
- A missing null check on optional data -> severity: minor

Skip:

- Naming style, import order, formatting

Example (critical): user input concatenated directly into a query string.

Example (minor): optional field read without a guard where default is safe.

What this shows: the report list, the skip list, and one example per severity together leave almost nothing for the model to guess, so classification stays consistent.

Common Mistakes

- Relying on being conservative instead of defining the actual boundary.
- Filtering by a confidence score the model is not calibrated to give.
- Defining severity tiers but giving no examples, so the line stays subjective.
- Leaving a noisy category running, which erodes trust in the accurate findings too.

References

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/increase-consistency>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

C. Few-Shot Prompting

Few-shot prompting means including a few completed examples of the task in your prompt before asking the model to handle a new case. A written rule describes what you want, but an example shows it. When the model can see two or three finished cases, it picks up the format, the level of detail, and the judgment calls all at once.

Few-Shot vs. Instruction-Only Prompting

Concept	Best Used For	Key Benefit	Common Exam Trap
Few-shot prompting	Consistent format, ambiguous cases, structured output	Strong consistency from concrete demonstration	Used to fix tool misrouting from weak tool descriptions
Zero-shot prompting	Simple, well-defined tasks	Low token cost	Expected to give consistent format on complex tasks

The deciding question is whether the correct output is easy to describe in words. If it is, zero-shot may be sufficient. If the correct output involves a specific format, a judgment call, or a distinction that is hard to articulate, few-shot will outperform any amount of additional instruction.

Why Examples Outperform Longer Instructions

Zero-shot prompting tells the model what to do. Few-shot prompting shows the model what "done" looks like. That difference matters more than it seems.

- Examples make the exact format concrete, so the model copies the structure instead of inventing one.
- Examples demonstrate judgment on borderline cases, which is hard to describe in rules but easy to show.
- Examples reduce made-up or empty fields in extraction because the model sees how a real, complete answer is built.

- Examples carry implicit information that would take many words to describe explicitly, such as tone, level of detail, and how to handle missing data.

Approach	What the Model Has to Do	Risk
Zero-shot	Imagine what a correct answer looks like.	Format varies, judgment calls are inconsistent
Few-shot	Copy the demonstrated pattern	Low, as long as examples are diverse and representative

Anatomy of an Effective Example

Anthropic's guidance is that good examples are relevant, diverse, and clear.

Quality	What It Means
Relevant	Each example looks like the real cases you will see, not a toy case.
Diverse	Examples cover different situations and edge cases, and differ enough that the model does not lock onto a pattern you did not intend.
Clear	Each example sits inside <code><example></code> tags; several examples grouped inside <code><examples></code> tags.

Key Principles for Example Design

- **Include reasoning:** Show the reason a choice was made, not just the final answer.
- **Demonstrate format:** Show the exact output shape you expect. The model will reproduce that shape.
- **Cover ambiguous cases:** Two to four examples that resolve genuinely tricky cases are worth more than many examples of the obvious case.
- **Show varied input formats:** In extraction work, show the same fact appearing in different document layouts.

How Many Examples Is Enough

- **0 (zero-shot):** the model invents the format and judgment from instructions alone with no reference point.
- **1 example:** better than zero but a single example risks creating an unintended pattern since there is only one case to generalize from.
- **3 to 5 examples:** Anthropic's documented sweet spot. Enough diversity to generalize correctly without adding excessive token cost.
- **More than 5:** diminishing returns on most tasks. Only worth the added cost when the goal is specifically to cover a wide range of edge cases.

Common Mistakes

- Using few-shot examples to fix tool misrouting when the real cause is a weak tool description.
- Making the examples too similar, so the model latches onto a pattern you did not intend.
- Showing only easy examples, leaving the ambiguous cases unguided.

- Leaving examples unwrapped, which makes them harder for the model to separate from instructions.

References

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/multishot-prompting>
 - <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/increase-consistency>
 - <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
-

D. Structured Output with Tool Use and JSON Schemas

Structured output means forcing the model's answer into a defined shape so the next system in line can read it without guessing. If a downstream service expects a field called `total_amount` holding a number, it needs that field, with that type, every single time. Free-form text cannot be relied on that way.

Tool Use

Tool use is the primary mechanism for getting structured output from Claude. You define a tool with a name, a description, and an input schema. Claude decides when to call it based on the request and the tool's description, and your code runs it.

How the Tool Use Loop Works

When a tool is available, Claude does not respond with plain text. It responds with a structured block indicating which tool to call and what inputs to pass. Your code takes that, runs the actual operation, and sends the result back. The loop continues until Claude has everything it needs to finish.

- Claude responds with `stop_reason tool_use` and one or more `tool_use` blocks containing the tool name and input arguments.
- Your code executes the operation and returns a `tool_result` to Claude.
- Claude continues generating based on the result, or returns `stop_reason end_turn` when finished.
- For extraction tasks, the tool acts as the output contract: its input schema defines the exact shape of the data you want returned.

Why Tool Descriptions Matter

The model decides whether to call a tool based on how the tool is described, not just on the instruction in the message. A vague or poorly written tool description leads to missed calls or wrong tool selection. Write tool descriptions with the same care as prompt instructions.

References

- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>
- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/implement-tool-use>

JSON Schema

A JSON schema is what turns a tool into an output contract. It declares exactly what fields the model must return, what type each field holds, and which fields are required. Without a schema, the model

decides the shape of the output on its own. With a schema, the shape is fixed.

Key Terms

- **JSON schema** is a formal description of fields, their types, which are required, and what values are allowed.
- **Structured output** is output forced into a defined, parseable shape.
- **Strict tool use** is `strict: true`, which guarantees the tool name and inputs match your schema.
- **Structured Outputs** is the native feature with two modes: JSON outputs and strict tool use.

JSON Schema Field Types

Field Type	Behavior	Use When
required	Must always be present; forces a value even if absent	Data the source always contains
optional	May be omitted when not in the source	Data that may be absent
nullable	May return null when value is absent	Data that may be absent but must appear in output
enum	Restricted to a fixed set of allowed values	Category fields with known values

Choosing the right field type is one of the most important decisions in schema design. The wrong choice does not break the schema, it silently produces bad data. A required field on something that is sometimes absent forces the model to invent a value. A missing nullable means the model fills a gap with a guess instead of returning null honestly.

- **required** is for data that will always exist in the source. If the field is required but the source does not contain it, the model has no honest answer to give, so it fabricates one. Only mark a field required when you are certain the source will always provide it.
- **optional** is for data that genuinely may not exist and where its absence is acceptable to the downstream system. The field simply does not appear in the output when the source has nothing to provide. Use this when a missing field is a valid state and the consumer can handle it.
- **nullable** is for data that may not exist but where the downstream system needs to see an explicit signal that it is missing, not just a silent omission. Returning null is an honest answer. It tells the consumer the field was looked for and not found, rather than leaving the consumer to wonder whether the field was skipped or never checked. This is the safer default for most optional data.
- **enum** is for fields where only a fixed set of values is valid. It prevents the model from inventing category labels or returning slight variations of the same value across runs. When using enum, always consider adding another option for values you have not anticipated, paired with a free-text detail field to capture what other means in that specific case.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/implement-tool-use>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

The tool_choice Setting

This setting controls whether the model calls a tool and which one. Getting this right is what separates a system that sometimes uses tools from one that always behaves predictably.

Key Term

- tool_choice** is the setting that controls whether and which tool the model calls.

The Four Modes

Mode	Behavior
auto	The model decides whether to call a tool. This is the default.
any	The model must call one of your tools, but it picks which.
tool	The model must call one specific tool that you name.
none	The model may not call any tool, even if tools are provided.

Choosing the Right Mode

- Use auto when the model should decide whether a tool is needed at all.
- Use any when you need structured output but the input type is unknown and multiple schemas are possible.
- Use tool when one specific step must always run first, such as extracting metadata before enrichment.
- Use none to test text-only behavior or to disable tools for a specific turn.

Syntax Errors vs. Semantic Errors

This distinction is heavily tested. A schema guarantees the shape of the output, but not the meaning of the values inside it. These are two separate problems requiring two separate solutions, and confusing them is one of the most common mistakes in production Claude integrations.

Concept	Best Used For	Key Benefit	Common Exam Trap
Syntax-error prevention (schema)	Guaranteeing valid, parseable structure	Removes malformed output	Assumed to also guarantee correct values
Semantic-error detection (validation)	Catching wrong or inconsistent values	Verifies meaning, not just shape	Expected to be handled by the schema alone

A schema stops broken JSON, but it will happily return line items that do not add up to the stated total, or a value placed in the wrong field.

Examples of Each Error Type

Error Type	Example	Caught by Schema?
Syntax error	Missing closing brace, wrong data type	Yes

Error Type	Example	Caught by Schema?
Semantic error	Line items do not sum to stated total	No
Semantic error	Value placed in the wrong field	No
Semantic error	Two fields that contradict each other	No

The table shows the boundary between what a schema can catch and what it cannot. Syntax errors are structural. The output is broken before you even look at the values, so the schema catches them immediately. Semantic errors are different. The output is structurally valid and the schema raises no complaint, but the values are wrong, misplaced, or contradictory. The problem only appears when you actually read and compare the values, which the schema never does.

Native Structured Outputs Feature

Beyond tool use, Claude offers a native Structured Outputs feature with two modes that can be used alone or together.

Mode	When to Use
JSON outputs (<code>output_config.format</code>)	When the final answer itself is the data, such as turning one document into one record.
Strict tool use (<code>strict: true</code>)	When the model is taking actions in an agentic flow and each action call needs to be schema-valid.

The key difference between the two modes is what you are trying to control. JSON outputs controls the shape of the final response, making it useful when the response itself is the deliverable. Strict tool use controls the inputs to each tool call, making it useful when the reliability of each action in a workflow matters. If the wrong data flows into a tool mid-pipeline, every step after it is built on a bad foundation. Strict tool use prevents that by rejecting any tool call whose inputs do not match the schema before it runs.

Common Mistakes

- Treating a schema as proof the values are correct. It only guarantees the shape.
- Marking data required when the source might not contain it, which forces the model to invent a value.
- Mixing up any (some tool, model chooses) with a tool (one specific named tool).
- Forgetting structure, which lets the inputs drift away from your schema.
- Ignoring truncation, where a low `max_tokens` cuts off an incomplete tool call.

References

- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>
- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/implement-tool-use>
- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

E. Schema Design for Reliable Extraction

How you design the fields in a schema has a direct effect on extraction accuracy. The same model, given a slightly different schema, will either invent a value or honestly report that something is missing. This section is about making the honest outcome the default.

Schema design is not just about structure. It is about giving the model a way to be accurate. A schema that marks everything required forces the model to produce values even when the source does not contain them. A schema that uses nullable fields gives the model an honest path: return null when the data is not there. The difference between these two approaches is the difference between fabricated data and reliable extraction.

Required vs. Optional vs. Nullable Fields

Choosing between these three is the most consequential decision in schema design. The wrong choice does not throw an error. It silently produces bad data.

Concept	Best Used For	Key Benefit	Common Exam Trap
Required field	Data the source always contains	Guarantees the field is present	Forces fabrication when the data is absent
Optional field	Data that may be absent	Lets the model omit it	Confused with nullable when you need an explicit null
Nullable field	Data that may be absent but must still appear as null	Prevents invented values	Left out, so the model fabricates instead

When to Use Each

- Use required only when you are certain the source will always provide the data. If there is any chance it will not, required is the wrong choice.
- Use optional when the downstream system can handle a missing field and treats absence and null the same way.
- Use nullable when the downstream system needs an explicit signal that the field was checked and nothing was found. A null value is an honest answer. A missing field is ambiguous.

Nullable vs. Optional

- Optional means the field can disappear from the output entirely.
- Nullable means the field must appear but can carry a null value.
- Nullable is the safer default for most absent data. It prevents fabrication while still giving the consumer a reliable signal that the field was looked for and not found.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

Enum Fields

An enum field restricts the model to a fixed set of allowed values. This eliminates invented category labels and prevents slight variations of the same value appearing across runs. Without an enum, a

category field might return **software** on one run and **Software Development** on the next, and **SaaS Tool** on another. With an enum, the output is always one of the values you defined, making downstream processing predictable and reliable.

Enum fields are most valuable in classification and categorization tasks where consistency matters more than flexibility. They are one of the simplest and most effective tools for reducing output variability.

The Closed Enum Problem

A closed enum is an enum with no fallback option. It works well when your defined values cover every case you will ever see. In practice, that is rarely true.

- A closed enum breaks when the source contains a value you did not anticipate. The model is forced to pick the closest match from the defined list, which may be wrong.
- The output looks valid because it passes schema validation, but the classification is incorrect.
- The failure is silent. There is no error, no warning, and no way to tell from the output that the model was forced into a bad choice.
- Over time, as new categories appear in your data, a closed enum produces more and more misclassifications without any visible signal that something is wrong.

The "Other" Plus Detail Pattern

The solution is to keep the enum open by adding another option paired with a free-text detail field. This is the recommended pattern when the category space is likely to grow or when the source data comes from outside your control.

- The enum keeps the output structured and consistent for known values.
- The other option gives the model an honest exit when a value does not fit any defined category.
- The detail field captures what other means in that specific case, preserving the information without breaking the structure.
- Downstream systems can process known categories automatically and route other values to a human review queue for reclassification.
- Over time, the detailed field values in other cases reveal which new categories are appearing frequently.

Representing Ambiguity

Add a value like **unclear** to give the model a way to flag genuinely uncertain cases. Without it, the model is forced to pick a category even when the source is ambiguous, which produces confident but wrong classifications.

- **unclear** is different from **other**. Other means a value exists but does not fit the list. Unclear means the source does not contain enough information to classify at all.
- Using unclear surfaces the uncertainty instead of hiding it behind a forced choice.
- Unclear cases can be routed to human review or flagged for follow-up, rather than silently entering the pipeline with a wrong classification.
- Tracking the volume of unclear results over time is also useful. A high rate of unclear often signals that the source data quality is low or that the enum categories need to be redefined.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

Format Normalization Rules

A normalization rule is prompt guidance that tells the model how to standardize a value before returning it. The schema fixes the shape of the output. Normalization rules fix the values inside that shape. Both are needed for clean extraction.

Why They Are Needed

Sometimes the schema shape is correct but the values inside it are inconsistent. Dates written in different formats, phone numbers with and without country codes, names in different cases. A schema cannot fix these because they are all valid strings that pass type validation. Normalization rules in the prompt fix them by telling the model exactly how to standardize values.

Without normalization rules, the same piece of information can appear in many different forms across documents:

- A date field might return **January 5, 2024** in one record, **01/05/24** in another, and **2024-01-05** in a third. All three pass schema validation. None of them are consistent enough for reliable downstream processing.
- A name field might return **john smith**, **John Smith**, and **JOHN SMITH** from three different source documents. The schema accepts all of them. The downstream system has to handle three versions of the same value.
- A currency field might return **\$1,200.00**, **USD 1200**, and **1200.00 USD**. Parsing these reliably requires extra logic that normalization rules eliminate at the source.

Types of Normalization Rules

Normalization rules fall into a few common categories depending on what kind of value needs standardizing.

Date and Time Rules

- Convert all dates to ISO 8601 format (YYYY-MM-DD). If a date is missing, return null. Do not infer a date.
- Convert all timestamps to UTC in ISO 8601 format (YYYY-MM-DDTHH:MM:SSZ).
- Do not infer or estimate a date from context. If the source does not contain it, return null.

Contact and Identity Rules

- Standardize phone numbers to E.164 format ([country code][number]). If a number is missing, return null.
- Return names in title case. Do not abbreviate first or last names.
- Return email addresses in lowercase. Do not correct or guess email addresses.

Currency and Numeric Rules

- Convert all currency values to the base unit as a number with two decimal places. Do not include currency symbols or thousands separators.

- Return percentages as decimal numbers (0.15 for 15%). Do not include the percent sign.
- Do not round values unless the source explicitly states a rounded figure.

Text and Category Rules

- Return all country names using ISO 3166-1 alpha-2 codes (US, GB, PH). Do not use full country names or abbreviations.
- Trim leading and trailing whitespace from all string fields.
- Return boolean fields as true or false only. Do not return yes, no, or other variations.

When Normalization Rules Are Not Enough

Normalization rules handle predictable inconsistencies. They do not handle all cases.

- If the source data is genuinely ambiguous, a normalization rule cannot resolve it. Use a nullable field and return null rather than guessing.
- If the same field appears in multiple formats within a single document, note this in the rule explicitly so the model applies the same standard to all instances.
- If the normalization rule is complex, consider adding a worked example to the prompt showing the before and after. Rules with examples are followed more reliably than rules stated as instructions alone.

Situation	Normalization Rule Handles It?	What to Do Instead
Date in multiple formats across documents	Yes	Add an ISO 8601 rule
Date that is genuinely absent	No	Use nullable, return null
Currency with different symbols	Yes	Add a base unit rule
Ambiguous value that could mean two things	No	Use unclear enum value or nullable
Whitespace and casing inconsistencies	Yes	Add trim and case rules

Where to Put Them

Put normalization rules in the prompt alongside the schema, not inside the schema itself. The schema declares the type. The prompt explains how to handle the value. Keep them close together so the model sees both at the same time.

Placing rules far from the schema increases the chance the model misses them in a long prompt. If you have many rules, group them by field type and place the group immediately after the schema definition.

Common Mistakes

- Marking data required when the source often will not contain it.
- Leaving out nullability, so the model fills the gap with a guess.
- Using a closed enum with no other for a category that keeps growing.

- Skipping normalization rules, which leaves messy values inside a valid shape.
- Treating optional and nullable as interchangeable when the downstream system needs an explicit null.
- Writing normalization rules without examples, which leads to inconsistent application on edge cases.
- Placing normalization rules far from the schema in a long prompt, where the model is less likely to apply them consistently.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

F. Validation, Retry, and Feedback Loops

A validation loop checks the model's output against your rules and, when it fails, sends the failure back so the model can try again. The most important judgment in this section is knowing whether a retry can actually succeed, because retrying the wrong kind of error wastes money and can produce a fabricated answer.

Validation is the layer between Claude's output and your downstream system. Without it, bad data passes through silently. With it, you catch failures early, correct what can be corrected, and flag what cannot. A well-designed validation loop does not just reject bad output. It tells the model exactly what went wrong and gives it a realistic chance to fix it.

What is a Validation Loop?

A validation loop is a process that checks the model's output against your defined rules and, when the output fails, sends the failure back to the model for correction. It is the layer between Claude's output and your downstream system. Without it, bad data passes through silently.

Why It Follows a Fixed Sequence

Each step in the loop depends on the one before it. Skipping a step does not just miss a check. It either lets bad data through to the next system or wastes a retry on an error that cannot be fixed.

Step	Action	What Happens If Skipped
1. Extract	Send the source document to Claude with the schema	No output to validate
2. Validate structure	Check output against the JSON schema	Malformed output reaches the downstream system
3. Validate semantics	Check values for consistency and correctness	Structurally valid but wrong data passes through
4. Classify the error	Determine if the error is resolvable or unresolvable	Unresolvable errors get retried, risking fabrication

Step	Action	What Happens If Skipped
5. Retry or escalate	Send a feedback prompt for resolvable errors and escalate the rest	Fixable errors never get fixed, unfixable errors loop indefinitely
6. Log the result	Record what failed, what was retried, and what the outcome was	Patterns in failures stay invisible

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Resolvable vs. Unresolvable Errors

This is the most important judgment in the section. Getting it wrong in either direction causes problems. Retrying a resolvable error without feedback just asks the model to guess again. Retrying an unresolvable error pushes the model toward fabrication.

What is a Resolvable Error?

A resolvable error is a format or structure problem where the information exists in the source document but came back in the wrong shape. The model found the data but returned it incorrectly, such as putting a value in the wrong field, returning a date in the wrong format, or producing malformed JSON. Because the data exists, a retry with specific feedback gives the model a realistic chance to fix it.

What is an Unresolvable Error?

An unresolvable error is a missing fact that no retry can produce. The information is simply not in the source document. If a field was marked required but the source never contained that data, the model has nothing to extract. Retrying does not create information that was never there.

Error Type	Cause	Correct Response
Resolvable	Information exists in the source but came back in the wrong shape	Retry with specific error feedback
Unresolvable	Information is not in the source at all	Do not retry. Adjust the input or schema instead.

The Fabrication Risk

When an unresolvable error is retried, the model knows it failed and is being asked to try again. Without the information it needs, it may produce a plausible-looking value to satisfy the requirement. That value is fabricated. It passes schema validation and looks correct. The only way to prevent this is to recognize unresolvable errors before retrying.

- **Fabricated value** is a value the model invented to satisfy a schema requirement when the actual data did not exist in the source. It looks valid, passes schema validation, and is wrong.
- **Plausible-looking** describes a fabricated value that resembles a real answer closely enough to pass automated checks. It is the reason fabrication is dangerous: it fails silently.

- **Silent failure** is when bad data passes through every validation layer and enters the downstream system without triggering any error or alert. Fabricated values are the most common cause of silent failures in extraction pipelines.

References

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>
- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

Retry with Error Feedback

A retry without feedback is just a second guess. The model does not know what went wrong, so it produces roughly the same output with minor variations. A retry with feedback is a targeted correction. The model knows exactly what failed and where to focus.

What to Include in a Feedback Prompt

On a retry, send back three things:

- The original input (the source document) so the model can re-read the source with fresh attention.
- The failed output (what the model returned) so the model can see exactly where it went wrong.
- The specific validation error (exactly what failed and why) so the model does not have to guess what needs to be fixed.

How to Write a Good Feedback Prompt

- Be specific about what failed. "invoice_number is missing" is more useful than "the output was invalid."
- Reference the exact field and the exact rule it violated. "total_amount must be a number but returned as a string" gives the model a precise target.
- Do not include errors the model cannot fix. If an unresolvable error is mixed in with resolvable ones, the model may fabricate a value for the unfixable field while correctly fixing the others.

Feedback Quality	Example	Result
No feedback	"Please try again"	Model guesses with no new information
Vague feedback	"The output was invalid"	Model makes minor random adjustments
Specific feedback	"invoice_number is missing. It appears in the header as INV-XXXX."	Model targets the exact field and location

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

Validation Layers

Validation is not a single check. It is a stack of checks, each catching a different class of error. Skipping a layer means that class of error passes through undetected.

What is a Validation Layer?

A validation layer is one level of checking in the validation stack. Each layer has a specific job and catches a specific class of error. No single layer catches everything. A schema catches structure problems. A semantic layer catches meaning problems. A business rule layer catches domain-specific logic problems. All three are needed in a production extraction pipeline.

Validation Layer	What It Catches	Tool
Schema validation	Structure, types, required fields, enum values	JSON schema, strict tool use
Semantic validation	Arithmetic consistency, field placement, contradictions	Pydantic, custom checks
Business rule validation	Domain-specific constraints	Custom logic

Schema Validation

Schema validation checks structure. It confirms that required fields are present, that values match their declared types, and that enum values are within the allowed set. This is the first layer and the cheapest to run.

- Catches missing required fields.
- Catches wrong data types.
- Catches values outside an enum.
- Does not catch wrong values, inconsistent values, or business rule violations.

Semantic Validation

Semantic validation checks meaning. It confirms that values are logically consistent with each other and with the rules of the domain. These are errors a schema cannot catch because the structure is valid.

- Line items that do not sum to the stated total.
- A value placed in the wrong field where the type still matches.
- Two fields that contradict each other.
- A date that is logically impossible given another date in the same record.

Business Rule Validation

Business rule validation checks domain-specific constraints that go beyond structure and arithmetic.

- A ship date must be after the order date.
- A discount cannot exceed the item price.
- A status of completed requires a non-null completion date.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

Semantic Validation Errors

These are logical errors a schema cannot catch. The output is structurally valid and passes schema validation, but the values are wrong or inconsistent. The schema confirmed the shape. It said nothing about whether the values make sense.

Why a Schema Cannot Catch These

A schema checks that a field exists, holds the right type, and falls within an allowed set. It does not compare values against each other. It does not recompute arithmetic. It does not know that a ship date before an order date is impossible. All of that logic has to live in a separate validation layer.

Common Semantic Errors

- **Line items that do not sum to the total.** Each line item field and the total field are individually valid numbers. The schema passes them both. Only a recomputation reveals the mismatch.
- **A value placed in the wrong field where the type still matches.** A name in an email field is still a string. A price in a quantity field is still a number. The schema accepts both. The error is only visible when you read the content, not the type.
- **Two fields that contradict each other.** A status of completed and a null completion date are individually valid. Together they are contradictory. The schema evaluates each field alone and never compares them.
- **A calculated field that does not match the stated value.** A subtotal that does not match the sum of its components, or a tax amount that does not match the declared rate applied to the base. Both fields are valid numbers. The inconsistency only appears when you do the math.

What Causes Semantic Errors

Semantic errors have two main causes. The first is extraction error, where the model correctly found the values but placed them incorrectly or copied the wrong number. This is often resolvable by retry with targeted feedback. The second is source inconsistency, where the source document itself contains contradictory or incorrect data. This is usually unresolvable by retry and should be escalated for human review.

Semantic Error	Likely Cause	Resolvable by Retry?
Line items do not sum to total	Model copied wrong number	Sometimes
Value in wrong field	Model placed value incorrectly	Yes, with specific feedback
Two contradicting fields	Source document inconsistency	Usually no
Calculated field mismatch	Model arithmetic error or wrong source value	Sometimes

Adding Semantic Checks to Pydantic

What Pydantic Does Here

Pydantic validates data against a defined model. Beyond type checking, it can run custom logic that catches the semantic errors a schema misses. When a check fails, Pydantic raises a `ValidationError` with a specific message. That message feeds directly into the retry prompt, giving the model a precise description of what went wrong.

Field Validators

A field validator runs custom logic on a single field after type validation passes. It can recompute a value and raise an error if the result does not match the stated value. For example, a field validator on a subtotal field can sum the line items and compare the result to the extracted subtotal.

Model Validators

A model validator runs after all individual field validators have passed. It has access to every field in the model at once, which makes it the right place for cross-field logic. A model validator can compare a ship date to an order date, check that a status and a completion date are consistent, or verify that a discount does not exceed the item price.

Why This Matters for the Retry Loop

Pydantic checks run automatically as part of the validation loop. When a check fails, the `ValidationError` it raises contains a specific message about which field failed and why. That message is what you include in the feedback prompt on a retry. A generic error message produces a generic retry. A Pydantic error message that says `line_item_total 150.00 does not match stated_total 180.00` points the model directly at the discrepancy.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

In-Schema Self-Checks

In-schema self-checks are fields you add to the schema specifically to surface inconsistencies. Rather than running all validation logic externally, you ask the model to do part of the checking itself and report the results as structured fields. This reduces the amount of external validation logic you need to write and makes certain classes of problems visible immediately in the output.

Why Add Self-Checks to the Schema?

External validation catches errors after the fact. Self-checks ask the model to flag potential issues as part of extraction. When the model detects a contradiction or expresses uncertainty during generation, capturing that signal in a structured field gives your validation layer something concrete to act on. Without self-checks, those signals are either lost or buried in free-form text that is harder to process reliably.

The Four Self-Check Fields

- `calculated_total` is a field where the model computes the total independently from the line items and returns it alongside the stated total from the source. A mismatch between the two

surfaces is an arithmetic inconsistency without requiring the validation layer to reparse and recompute the line items externally.

- **conflict_detected** is a boolean field the model sets to true when it finds contradictory information in the source document that it cannot resolve on its own. For example, a document that states a quantity of ten in one section and five in another. The model cannot know which is correct, so it flags the conflict and lets a human decide.
- **detected_pattern** is a field that records which construct, condition, or rule triggered a finding. Over time, tracking this field across many documents reveals which criteria produce the most false positives, which categories fire most often, and where the schema or prompt criteria need tightening.
- **confidence** is a field where the model signals how certain it is about a particular extraction. A low confidence score on a field means the model found something but is uncertain whether it is correct. Routing low-confidence records to human review before they enter the downstream system prevents uncertain extractions from propagating silently.

Self-Check Field	What It Surfaces	How to Use It
calculated_total	Arithmetic mismatches	Compare against stated_total in validation layer
conflict_detected	Source-level contradictions	Route to human review queue
detected_pattern	Which rule triggered a finding	Track over time to identify noisy criteria
confidence	Model uncertainty	Route low-confidence records to review

Important Terms

- **Self-check field** is a schema field whose purpose is not to capture extracted data but to capture the model's own assessment of the extraction, such as a computed total, a conflict flag, or a confidence score.
- **stated_total** is the total as it appears in the source document, extracted directly. It is compared against **calculated_total** to detect arithmetic mismatches.
- **Boolean field** is a field that holds only true or false. **conflict_detected** is a boolean because the model either detected a conflict or it did not. There is no middle value.
- **Human review queue** is a holding area where records are sent when automated validation cannot resolve an issue. Records with **conflict_detected** set to true or with a low confidence score are routed here rather than being retried or dropped.
- **Noisy criteria** are rules or conditions that trigger findings too frequently on cases that are not actual problems. The **detected_pattern** field is what makes them visible over time so they can be tightened or removed.
- **False positive rate** is the proportion of findings raised by a criterion that turn out not to be real problems. A high false positive rate on a specific pattern is the signal that **detected_pattern** is designed to surface.
- **Confidence score** is a value the model returns alongside an extraction to indicate how certain it is that the extracted value is correct. Low confidence does not mean the value is wrong, but it means a human should verify it before the record enters production.
- **Propagation** is when an incorrect value enters the downstream system and flows through subsequent processing steps unchecked. Self-checks exist to catch propagation at the source before it compounds.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Max Retry Limits

What is a Max Retry Limit?

A max retry limit is the maximum number of times the validation loop will attempt to correct a failed output before stopping and escalating the record. Without a limit, the loop runs indefinitely on unresolvable errors, consuming tokens and time without any chance of succeeding. Setting a limit is not optional in production systems. It is a safety boundary that prevents runaway costs and infinite loops.

Why Limits Are Necessary

A retry loop that has no exit condition assumes every error is eventually fixable. That assumption is wrong. Unresolvable errors cannot be fixed by any number of retries because the information simply does not exist in the source. Without a limit, the loop keeps sending the same unresolvable error back to the model, which responds by fabricating increasingly plausible-looking values just to satisfy the requirement. The output looks correct, passes validation on a later attempt by chance, and enters the downstream system as fabricated data.

Setting the Right Limit

- Set a max retry limit of two to three attempts for most extraction tasks.
- A single retry handles most resolvable errors, such as a formatting mistake or a misplaced value. A second retry catches cases where the first retry introduced a new minor error.
- Going beyond three retries rarely improves accuracy and significantly increases cost. If an error persists after three attempts, it is almost always unresolvable.
- Set different limits for different task types. Simple field extraction may need only one retry. Complex multi-document extraction may justify three.

What Happens When the Limit Is Reached

- Log the failure with the source document, the last output, and all validation errors. This creates a record that can be reviewed later to understand what went wrong.
- Route limit-exceeded records to a human review queue rather than dropping them silently. Dropping records creates invisible data loss. A human review queue makes the failure visible and actionable.
- Do not let limit-exceeded records enter the downstream system, even if the last output looks structurally valid. A record that required three retries and still failed is not reliable.

Using the Retry Limit as a Diagnostic Signal

- Track which fields and which document types hit the retry limit most often. A field that consistently hits the limit is a signal that the schema, the prompt, or the source data for that field has a systemic problem.
- A document type that regularly exhausts retries suggests that the prompt examples do not cover that format or that the schema is too strict for the variation in that document class.

- The retry limit count per field over time is one of the most useful metrics in an extraction pipeline. It shows exactly where the system is failing and where schema or prompt improvements will have the most impact.

Situation	What It Signals	What to Do
One field hits the limit repeatedly	Schema or prompt problem for that field	Review the field definition and prompt criteria
One document type hits the limit repeatedly	Format not covered by examples	Add examples for that document type
All fields hit the limit on the same document	Source document is unusable	Escalate for human review, flag the source
Limit is hit on the first retry attempt	Error is likely unresolvable from the start	Classify the error before retrying

Important Terms

- **Retry limit** is the maximum number of correction attempts allowed before the loop stops. It is a hard boundary, not a suggestion.
- **Runaway loop** is a retry loop with no exit condition that keeps running indefinitely on an unresolvable error, consuming tokens and producing fabricated values with each attempt.
- **Limit-exceeded record** is a record that has reached the max retry limit without passing validation. It must be logged and routed to human review, never dropped silently.
- **Systemic problem** is a recurring failure that appears across many documents rather than on a single edge case. A field that repeatedly hits the retry limit is a signal of a systemic problem in the schema or prompt, not a one-off extraction error.
- **Silent data loss** is when a record is dropped without logging after hitting the retry limit. It is one of the most dangerous failure modes in an extraction pipeline because nothing signals that the record is missing.
- **Token cost** is the number of tokens consumed by each API call. Every retry consumes tokens for the full prompt plus the output. Without a retry limit, unresolvable errors can consume many times the expected token budget before failing.

WORKED EXAMPLE

```
from pydantic import BaseModel, ValidationError, model_validator

class Invoice(BaseModel):
    invoice_number: str
    line_item_total: float
    stated_total: float

    @model_validator(mode='after')
    def check_totals_match(self):
        if round(self.line_item_total, 2) != round(self.stated_total, 2):
            raise ValueError(
                f"line_item_total {self.line_item_total} does not match "
```

```

        f"stated_total {self.stated_total}"
    )
    return self

MAX_RETRIES = 3

def validate_and_retry(raw_output, document, call_model, attempt=1):
    try:
        return Invoice.model_validate_json(raw_output)
    except ValidationError as e:
        if attempt >= MAX_RETRIES:
            raise RuntimeError(f"Max retries reached. Last error: {e}")
        return validate_and_retry(
            call_model(document=document, failed=raw_output, error=str(e)),
            document, call_model, attempt + 1
        )

```

What this shows: the model validator catches the semantic error that schema validation misses, the feedback prompt sends the exact mismatch back to the model, and the max retry limit prevents indefinite looping on unresolvable cases.

Common Mistakes

- Retrying when the data does not exist, which pushes the model toward fabrication.
- Expecting the schema to catch sums or field placement. Those are semantic checks.
- Retrying without the specific error, so the model just guesses again.
- Skipping the tracking field, which leaves failure patterns invisible.
- Setting no retry limit, which causes indefinite looping on unresolvable errors.
- Mixing resolvable and unresolvable errors in the same feedback prompt, which can trigger fabrication for the unfixable fields.
- Not logging retry outcomes, which makes it impossible to identify systemic schema or prompt problems.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

G. Batch Processing Strategies

Batch processing is for when you can accept a delay in exchange for paying half the price. That makes it a good fit for large volumes of work that can wait, and a poor one for anything where someone is sitting there waiting on the result.

The Message Batches API is built for exactly that kind of workload, where cost and volume matter but latency doesn't. This section covers when to reach for it, how the results come back, and how to handle failures along the way.

How the Message Batches API Works

Submission

You submit a batch as a single API call containing an array of requests. Each request carries its own `custom_id`, `model`, `max_tokens`, and `messages`. The API accepts the batch and begins processing asynchronously. Your system does not wait for results.

Processing Window

The Message Batches API guarantees processing within 24 hours. Most batches complete in under an hour, but your system must be designed to handle the full window. Any SLA built on batch processing must absorb this uncertainty.

Polling for Results

Because results are not returned at submission time, your system must poll the API to detect when the batch is complete. Poll on a reasonable interval, such as every few minutes, rather than continuously. When the batch status changes to completed, retrieve the results.

Result Retrieval

Results do not come back in the same order as the input requests. Each result carries the `custom_id` of the request it belongs to. Matching results back to their source documents is done by `custom_id`, not by position.

Phase	What Happens	What Your System Does
Submission	Batch accepted, processing begins	Record the batch ID and submission time
Processing	API processes requests asynchronously	Poll periodically for status
Completion	Results available for retrieval	Fetch results, match on <code>custom_id</code>
Failure handling	Some items failed or expired	Identify failed items by <code>custom_id</code> , resubmit

References

- <https://docs.claude.com/en/docs/build-with-claude/batch-processing>
- <https://www.anthropic.com/news/message-batches-api>

Batch vs. Synchronous Processing

The choice between batch and synchronous is not about preference. It is about what the work actually needs. Choosing the wrong processing mode either blocks a workflow that cannot afford to wait or wastes money on real-time infrastructure for work that has no urgency.

What is Batch Processing?

Batch processing is a mode where you submit many requests at once and retrieve results later. The API processes them asynchronously in the background. You do not wait for each result before moving on.

Because you are accepting a delay, Anthropic charges 50% of the standard synchronous price. The trade is explicit: time for money.

What is Synchronous Processing?

Synchronous processing is the standard API mode where each request waits for a response before proceeding. The result comes back in the same call. It is required whenever something is blocked waiting on the answer, whether that is a person on screen, a pipeline step, or an automated check that must complete before the next action.

Concept	Best Used For	Key Benefit	Common Exam Trap
Batch processing	Overnight reports, weekly audits, bulk evaluation, document processing	50% cost reduction at scale	Applied to a blocking workflow that needs a fast result
Synchronous processing	Pre-merge checks, interactive flows, agentic tool loops	Immediate response	Used for huge non-urgent volumes where cost matters

The Deciding Question

Ask: does anything stop working while waiting for this result? If yes, use synchronous. If no, use batch. The cost saving is irrelevant if the workflow is blocked.

What Batch Processing Cannot Do

- It cannot guarantee when results will arrive. There is no latency SLA beyond the 24-hour window.
- It cannot support multi-turn tool loops within a single request. Agentic workflows must use the synchronous API.
- It cannot be used when downstream steps depend on the result immediately.
- It cannot replace synchronous processing when an SLA requires results faster than 24 hours.

Choosing the Right Mode

Workload	Right Choice	Reason
Overnight document extraction	Batch	Latency-tolerant, high volume, cost matters
Pre-merge code review	Synchronous	Developer is waiting for the result
Weekly audit of 10,000 records	Batch	No urgency, significant cost saving
Interactive chatbot response	Synchronous	User is waiting on screen
Agentic tool loop	Synchronous	Multi-turn exchange required
Monthly billing report	Batch	Overnight run, latency irrelevant
Pipeline step where next step is blocked	Synchronous	The downstream step cannot proceed without the result
End-of-day summary generation	Batch	Consumed the next morning, no urgency

The table shows one principle applied across different workloads: the right choice is always determined by who or what is waiting, not by volume. Every synchronous case has something blocked waiting on the result. Every batch case has nothing waiting. Volume does not drive the decision. Whether something is blocked does.

References

- <https://docs.claude.com/en/docs/build-with-claude/batch-processing>
- <https://www.anthropic.com/news/message-batches-api>

Failure Handling

Failures in batch processing are expected at scale. The key is identifying which items failed, why they failed, and how to resubmit them correctly. A batch that partially fails is not a failed batch. It is a normal outcome that your system should be designed to handle.

Custom Identifiers

A custom identifier, or `custom_id`, is a unique label you assign to each request before submitting it to the batch. Its purpose is to let you match every result back to the document or record it came from after processing is complete. Without it, results are anonymous. You receive a set of outputs with no reliable way to know which output belongs to which input.

The `custom_id` is necessary because batch results do not come back in the same order as the input requests. The API processes items asynchronously, and the order of completion depends on processing time, not submission order. Position-based matching breaks entirely in this situation. `custom_id`-based matching works regardless of order.

- Assign a `custom_id` that ties directly to the source record, such as a document ID or a database primary key. This makes matching unambiguous and removes the need for a separate lookup table.
- Store the mapping between `custom_id` and source record before submitting. If the batch system fails entirely, you need this mapping to reconstruct what was submitted and what still needs processing.
- `custom_id` values must be unique within a batch. Duplicate IDs produce unpredictable matching behavior because two results will carry the same identifier with no way to tell them apart.

Resubmission

Resubmission is the process of sending failed items back to the API for a second attempt. When some items in a batch fail, only those items need to go back. Resubmitting the whole batch means processing items that already succeeded a second time, which doubles the cost for those items with no benefit.

Selective resubmission by `custom_id` keeps the cost of failure handling proportional to the number of failures, not the size of the original batch. If 100 items out of 10,000 fail, only those 100 are resubmitted. The other 9,900 are already done.

Chunking

Chunking is splitting a request that is too large into smaller pieces so it can be processed within the API's size limits. An item fails with a size error when the content of that single request exceeds what the API

will accept. Retrying the item as-is will fail again for the same reason. The item must be broken into smaller chunks first.

Each chunk is submitted as its own independent request with its own `custom_id`. Because each chunk returns its own result, your system needs to reassemble the chunk results into the complete output after resubmission. A chunk that is still too large after the first split must be split again until every piece falls within the size limit.

Batch Expiration During Busy Periods

Batch expiration is when the API does not finish processing a batch or individual items within it before the processing window closes, typically because of high system load. An expired item has not been processed at all. It must be resubmitted from scratch.

Expiration is more likely when a single very large batch is submitted during a high-load period because the API has a greater volume of work to process before reaching your requests. Splitting one large batch into several smaller batches reduces this risk. Smaller batches are lighter loads for the API to absorb and are more likely to complete within the processing window than one very large submission.

Failure Type	Cause	Correct Response
Item returned an error	Processing error on that request	Resubmit the item with the same <code>custom_id</code>
Item expired	API load prevented processing in time	Resubmit the item, consider splitting the batch
Item too large	Request exceeded size limits	Chunk the item and resubmit the chunks
Entire batch expired	Extreme API load	Resubmit the batch, split into smaller batches

Reference

- <https://docs.claude.com/en/docs/build-with-claude/batch-processing>

Submission Frequency and SLA Constraints

An SLA, or Service-Level Agreement, is a promise about when results will be delivered. Honoring an SLA in batch processing requires planning around the full 24-hour processing window, not just the typical completion time. A batch submitted at 11pm with a 9am delivery deadline has only ten hours of headroom. If the batch takes the full 24 hours, the deadline is missed.

Processing Window

The processing window is the period the API takes to complete a batch. Most batches finish in under an hour. However, the guaranteed maximum is 24 hours. Batches expire if processing does not complete within that window. Your submission timing must absorb the worst case, not the typical case.

Batch results are available for retrieval for 29 days after the batch is created. After 29 days, results can no longer be downloaded even if the batch itself is still visible.

Buffer Time

Buffer time is the additional time built into a submission schedule to absorb the full 24-hour processing window and leave room for at least one resubmission before the deadline. A schedule with no buffer has no recovery path when items fail or when processing takes longer than expected.

First-Pass Success Rate

The first-pass success rate is the proportion of items that return a valid result without resubmission. A higher rate means fewer resubmissions and more time to meet the deadline. The Anthropic documentation recommends testing a single request shape with the Messages API before submitting a full batch to avoid validation errors that would cause widespread failures. A 10% failure rate on 100,000 documents is 10,000 resubmissions. The same rate on 100 documents costs almost nothing to fix.

Planning Your Schedule

- Submit early enough that a full 24-hour processing time still meets the deadline.
- Build in a resubmission window. A schedule with no recovery path for failures will breach the SLA when items fail.
- Test a single request with the Messages API before large submissions to catch validation errors early.
- Break very large datasets into multiple batches. A single batch is limited to 100,000 requests or 256 MB, whichever comes first.

Submission Timing	Risk	Mitigation
Submitted too close to the deadline	Full 24-hour window breaches SLA	Submit earlier, build in buffer time
Large volume with untested prompt	Widespread validation errors, expensive resubmission	Test one request with the Messages API first
Single large batch during busy periods	Higher expiration risk	Split into smaller batches
No resubmission window in the schedule	Failed items breach the SLA	Always plan for resubmission before submitting

The table covers the four most common submission mistakes and their fixes. In each case the risk is not just a failed batch but a missed deadline or a resubmission cost that could have been avoided. Submitting early, testing before scaling, splitting large batches, and building in a resubmission window are not optional best practices. They are the minimum requirements for a batch workflow that reliably meets its SLA.

Common Mistakes

- Batching a blocking workflow. There is no latency guarantee.
- Assuming results come back in order instead of matching on `custom_id`.
- Expecting a tool loop inside a single batch request. This is not supported.
- Resubmitting the whole batch instead of only the failed items.
- Submitting a large volume without testing the prompt on a sample first.

- Not accounting for the full 24-hour window when planning against an SLA.
- Using duplicate custom_id values within a batch.

References

- <https://docs.claude.com/en/docs/build-with-claude/batch-processing>
- <https://www.anthropic.com/news/message-batches-api>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

H. Multi-Instance and Multi-Pass Review

Multi-instance and multi-pass review are two complementary patterns that address the predictable weaknesses of a single all-in-one review. Multi-instance review uses separate, independent Claude instances so that generation and review are never done by the same session. Multi-pass review splits a large review into focused passes so that each pass has a narrow, well-defined job. Used together, they produce reviews that are more consistent, more thorough, and harder to fool than any single-instance, single-pass approach.

Self-Review vs. Independent Review Instance

Concept	Best Used For	Key Benefit	Common Exam Trap
Self-review (same session)	Quick sanity checks	Low overhead	Trusted to catch subtle issues it generated
Independent review instance	Catching subtle defects in generated output	Fresh context, no generation bias	Overlooked in favor of "review your own work"

The self-review limitation: A model that just generated an output still holds the reasoning it used to produce it, so it is less likely to question its own decisions in the same session.

Multi-Pass Review

Pass Type	Best Used For	Key Benefit	Common Exam Trap
Per-file local pass	Local bugs and within-file issues	Consistent depth per file	Expected to catch cross-file issues
Cross-file integration pass	Data flow and interaction across files	Detects integration defects	Skipped, leaving integration bugs unfound

Together, the two passes fix the common failure where a single pass gives deep feedback on some files, shallow feedback on others, and even contradicts itself on identical code.

Confidence-Based Review Routing

- Have the model report a confidence level with each finding.
- Send low-confidence findings to closer review.
- This focuses on limited reviewer time where it is most needed.

WORKED EXAMPLE

Phase 1 (per file): For each changed file, review it in isolation for local bugs, security issues, and error handling.
Output: location, issue, severity, confidence.

Phase 2 (integration): Using a fresh instance, review how the changed files interact: shared state, data flow, and contract mismatches across files.

Routing: send any finding with confidence below the threshold to human review.

What this shows: Phase 1 keeps depth consistent across files, Phase 2 catches the cross-file defects, and confidence routing puts human attention on the least certain findings.

Common Pitfalls

- Trusting same-session self-review to catch subtle defects.
- Running a single pass over many files, which spreads attention too thin.
- Skipping the integration pass, which misses cross-file bugs.
- Requiring agreement across repeated full runs, which can hide real bugs caught only some of the time.

QUICK REFERENCE

- Subtle defects: use an independent instance.
- Many files with inconsistent feedback: use per-file passes plus an integration pass.
- Limited reviewer time: route by confidence.
- Do not rely on a larger context window or on full-run agreement.

EXAM TIPS:

- Remember that an independent instance beats same-session self-review for subtle defects.
- The exam may test scenarios where a single pass over many files gives inconsistent feedback.
- Choose the solution that splits review into per-file passes plus a cross-file integration pass.
- Avoid options relying on a larger context window or agreement across repeated full runs.

References

- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/increase-consistency>

Domain 4 Services Appendix

Claude API

API Element	What It Is	Why It Matters	EXAM TIP
tool_use	A content block where the model requests a defined function call, returned with stop_reason tool_use.	The mechanism for actions and structured data extraction. Your code executes client tool calls and returns a tool_result for the next turn.	—
tool_choice	The parameter controlling whether and which tool is called. Modes: auto, any, tool, none.	Deterministic control over tool calling.	auto is the default. Do not confuse any with tool.
stop_reason	The field indicating why the model stopped (tool_use or end_turn).	Drives the control flow of an agentic loop.	Terminate loops on stop_reason, not on parsing the text.
max_tokens	The maximum tokens the model may generate.	Too low a value can truncate output, including a tool call.	A truncated tool call usually means: raise max_tokens and retry.
system prompts	Top-level instructions setting role, rules, and output expectations.	Wording here can influence tool selection and behavior.	Keyword-sensitive wording can override good tool descriptions.

JSON Schema Field Reference

Field Type	Behavior
required	Always present. Forces a value even when absent from the source.
optional	May be omitted when not in the source.
nullable	May be returned as null. Prevents fabrication.
enum	Restricts a value to a defined set.
"other" + detail pattern	Another enum option paired with a free-text detail field for extensible categories.

Pydantic

- Validates Claude's output against expected types and constraints.
- Surfaces semantic errors that schema-shape enforcement misses.
- On failure, drives a validation-retry loop using the original input, the failed output, and the specific error, when the data exists in the source.

Message Batches API

Property	Value
What it is	Asynchronous processing of many requests together.
When to use	Latency-tolerant work: overnight reports, audits, and bulk evaluation.
Cost savings	50% of standard API prices.
Processing window	Within 24 hours. Most batches finish in under an hour.
custom_id usage	A unique identifier per request.
Result correlation	Results are not in input order. Match on custom_id.
Failure handling	Resubmit failed items only. Chunk oversized documents.
Not for	Latency-sensitive work (no latency guarantee) or multi-turn tool loops.

Few-Shot Prompting

- Provide 3 to 5 diverse, relevant, clear examples in `<example>` tags.
- Improves consistency for structured and format-sensitive tasks.
- Demonstrates format and reasoning and handles ambiguous, varied inputs.

Prompt Chaining

- Break a complex task into a sequence of focused steps, where each step's output feeds the next.
- Improves reliability by giving each step a narrower job, reducing attention dilution.
- Example: review each file individually, then run a separate cross-file integration step.

Domain 4: Prompt Engineering & Structured Output — Sample Questions

Question 1

A company uses an AI review tool to provide feedback on submitted work.

Users report that the tool incorrectly flags items as unused or invalid because it does not recognize an indirect usage pattern. The feedback also uses inconsistent formats. Some findings follow a clear structure, while others are written as unstructured paragraphs.

The company needs to improve both the accuracy of the review and the consistency of the output format.

Which combination of techniques is most effective?

1. Add detailed instructions that explain the indirect usage pattern and specify the exact output format for each finding.
2. Add few-shot examples that show how to correctly handle the indirect usage pattern and demonstrate the exact finding format, including location, issue, severity, and suggested fix.
3. Add a post-processing linter to validate the output format and a separate analysis step to resolve indirect usage before review.

4. Create explicit rules for every possible indirect usage pattern and require all findings to follow a strict JSON output schema.

Correct Answer: 2

Explanation:

Few-shot prompting is useful when an AI review tool must learn both what to identify and how to present the result. In this scenario, the tool is making accuracy errors because it does not understand an indirect usage pattern, and it is also producing inconsistent feedback formats. A few-shot prompt can show the model realistic examples of acceptable indirect usage, incorrect findings that should be skipped, and valid findings that should be reported. Anthropic's prompting guidance recommends using examples to improve performance, especially when the task requires consistent judgment or a specific response pattern.

The best solution is to provide examples that demonstrate both the review logic and the required output format. For example, one sample can show an item that looks unused but is actually referenced indirectly, so it should not be flagged. Another sample can show a real issue with a properly formatted response that includes location, issue, severity, and suggested fix. This approach gives the model concrete patterns to follow rather than relying only on abstract instructions.

This also improves output consistency because the examples act as a template for the final response. A strict schema or post-processing step can help enforce formatting, but examples help the model understand the intended review behavior before output generation. For review workflows, few-shot examples are often more practical than trying to write exhaustive rules for every possible edge case, especially when the tool must handle varied indirect patterns and still produce predictable findings.

Hence, the correct answer is: **Add few-shot examples that show how to correctly handle the indirect usage pattern and demonstrate the exact finding format, including location, issue, severity, and suggested fix.**

The option that says: *Add detailed instructions that explain the indirect usage pattern and specify the exact output format for each finding* is incorrect because it simply describes the desired behavior without showing concrete examples. Instructions are helpful, but examples are typically stronger when the model must apply a nuanced review pattern and follow a consistent format.

The option that says: *Add a post-processing linter to validate the output format and a separate analysis step to resolve indirect usage before review* is incorrect because it primarily adds external controls after or around the model output. This can help catch formatting problems and improve static analysis, but it does not directly teach the review tool how to reason about the indirect usage pattern and produce the expected response.

The option that says: *Create explicit rules for every possible indirect usage pattern and require all findings to follow a strict JSON output schema* is incorrect because it typically becomes difficult to maintain and may still miss new or unusual patterns. A strict schema can enforce structure, but it does not by itself teach the model when a finding is valid or when an apparent issue should be skipped.

References:

- <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices#use-examples-effectively>
- <https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/increase-consistency>
- <https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools>

Question 2

Your classifier labels contract clauses as termination, payment, or liability. It frequently mislabels force majeure clauses as termination because both describe conditions that end obligations. Positive examples alone have not resolved the confusion.

What prompt technique most directly tightens the classification boundary?

1. Removing all examples from the prompt to force the model to rely on label definitions rather than pattern-matching prior examples.
2. Adding counter-examples that explicitly show force majeure clauses labeled as incorrect classifications with a brief explanation of why.
3. Increasing the number of positive examples for the termination label to reinforce the correct pattern more strongly.
4. Switching to a longer prompt that describes each label in prose paragraphs rather than using structured examples and definitions.

Correct Answer: 2

Explanation:

When a classifier confuses two labels that share surface features, both force majeure and termination clauses describe conditions that end obligations, positive examples cannot resolve the confusion on their own. Positive examples demonstrate what a label looks like; they don't show where it stops. The model fills that gap with inference, and in boundary cases the inference is wrong.

Counter-examples address the boundary directly. They show inputs that resemble the target class but belong elsewhere, paired with a short rationale that names the distinguishing feature. Anthropic's prompting best practices recommend that examples be relevant, diverse, and structured, and explicitly call out diversity as covering edge cases so the model doesn't pick up unintended patterns — which is precisely the failure mode here. Wrapping each case in `<example>` tags (and the full set in `<examples>`) is the recommended structure.

Hence, the correct answer is: **Adding counter-examples that explicitly show force majeure clauses labeled as incorrect classifications with a brief explanation of why.**

The option that says: *Removing all examples from the prompt to force the model to rely on label definitions rather than pattern-matching prior examples* is incorrect. Removing examples typically degrades classification accuracy across all inputs, not just boundary cases. Examples are the most effective mechanism for demonstrating classification behavior. Prose definitions alone are less precise, and removing examples eliminates the annotated demonstrations the model needs to distinguish overlapping labels.

The option that says: *Increasing the number of positive examples for the termination label to reinforce the correct pattern more strongly* is incorrect because the model already handles clear termination

clauses correctly. The failure simply occurs at the boundary between termination and other labels, a boundary that more positive examples of the correct class cannot define. Only counter-examples showing what termination is not can tighten that boundary.

The option that says: *Switching to a longer prompt that describes each label in prose paragraphs rather than using structured examples and definitions* is incorrect because prose descriptions add length without adding the annotated demonstrations that make boundaries concrete. A longer definition of termination does not show the model where termination ends and force majeure begins. Counter-examples with rationales achieve that precision; prose paragraphs do not.

References:

- <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
 - <https://claudecertifications.com/claude-certified-architect/domains/prompt-engineering>
-

References for Domain 4: Prompt Engineering & Structured Output

All links reference official Anthropic documentation.

Prompt Engineering Overview

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/be-clear-and-direct>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/multishot-prompting>

Tool Use with Claude

- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>
- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/implement-tool-use>

Structured Output

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

Batch Processing

- <https://docs.claude.com/en/docs/build-with-claude/batch-processing>

Introducing the Message Batches API

- <https://www.anthropic.com/news/message-batches-api>